LOCAL-FIRST PRESENTATION PRODUCTION

Presentation Studio Source Installation Guide

Set up the Electron desktop app, connect any standard MCP client, and understand the project and export security boundaries.

INSTALLATION	PROJECTS	AI CONTROL
One-line source install macOS and Windows	Self-contained optional encryption	Local MCP human review gates

Important This release uses a one-line source installer, not an unsigned DMG, PKG, MSI, EXE, or app-store package. It installs prerequisites and verifies the app without disabling operating-system protections.

Version 0.3.1 - August 2026

Before you begin

Presentation Studio is designed for local presentation review and redesign. Imported PowerPoint files are copied into a self-contained project, audited without changing the originals, and exported only as new files through a human-controlled save or export action.

Requirements

Operating system	macOS or Windows
Runtime	Checked and installed automatically when Node.js 22.13 or newer with npm is unavailable
Disk space	Enough local space for source decks, packaged Resources, recovery files, and exported copies
Network	Needed during installation; the running app blocks non-local network requests

Install Version 0.3.1 on macOS

Paste this complete line into Terminal. It does not require Git.

```
curl -fsSL https://raw.githubusercontent.com/adammalin/Presentation-Studio/codex/web-slide-design-engine/scripts/install-macos.sh | /bin/zsh
```

Install Version 0.3.1 on Windows

Paste this complete line into PowerShell. It does not change execution policy.

```
irm https://raw.githubusercontent.com/adammalin/Presentation-Studio/codex/web-slide-design-engine/scripts/install-windows.ps1 | iex
```

Run again to update	Close Presentation Studio, then run the same one-line command. The installer verifies a staged copy before replacing the current managed app and keeps one previous managed copy.
Data boundary	Do not place client decks, manuscripts, project packages, extracted text, previews, or exports in the installation source folder. Projects and exports remain outside the managed app source.

What the installer does

1. Checks the operating system, processor architecture, install location, Node.js, and npm.
2. When needed, downloads the official portable Node.js 22.13 runtime and verifies it against the official SHA-256 manifest.
3. Downloads Presentation Studio 0.3.1 from the isolated release branch without requiring Git.
4. Runs the locked dependency install, automated tests, repository data-safety scan, and production renderer build in a staging folder.
5. Registers the installed MCP server automatically when a local Codex configuration is detected; restart Codex afterward.
6. Activates the verified app, creates a reusable launcher, and starts Presentation Studio.

Managed locations

macOS	~/Applications/Presentation Studio	Launch Presentation Studio.command
Windows	%LOCALAPPDATA%\Presentation Studio	Launch Presentation Studio.cmd

Microsoft PowerPoint PowerPoint is optional for launching the app but required for PowerPoint-native rendering and final native validation. A local PDF.js rasterizer is included, so Poppler, pdftoppm, and Homebrew are not required. Licensed Microsoft software is not installed automatically.

System security The installers do not alter Gatekeeper, quarantine settings, PowerShell execution policy, SmartScreen, or other protections. Follow your organization's approved software process if execution is blocked.

Developer checkout

Developers who need Git history can use the manual checkout procedure in README.md. The one-line installer is the default for ordinary users.

Connect an MCP client

Presentation Studio exposes a standard STDIO MCP server. It can be used by any compatible AI client; the app does not contain a provider-specific model or API-key workflow.

1. Open Presentation Studio and keep the desktop app running.
2. For Codex, the one-line installer registers the installed server automatically. Restart Codex after installing or updating.
3. For another MCP client, print the standard configuration snippet with the command below and add the presentation-studio entry to its normal mcpServers configuration.
4. Turn on the visible AI access switch in Presentation Studio when you want the current project, embedded Resources, and deck context available.

```
node scripts/configure-mcp.mjs
```

To merge the entry into an explicitly selected JSON file:

```
node scripts/configure-mcp.mjs --write /absolute/path/to/mcp-config.json
```

MCP safety model

Allowed	Human only
Check app status Read authorized deck context Stage semantic design changes Build private local candidates Record qualification evidence	Confirm a template exception Apply or reject a legacy proposal Save a project Export a PowerPoint copy Distribute an output

Current MCP boundary Authorized MCP models can inspect, stage semantic design changes, build private local candidates, and record qualification evidence. They cannot overwrite an original, save the project, export to a user destination, or distribute an output.

Local connection The STDIO server reaches the active app through a per-session token on a loopback-only bridge. The token descriptor is written with private local permissions and removed when the app closes.

Start a ChatGPT Desktop design session

Use this after Presentation Studio is open, the MCP connection is enabled, and the approved project material is in place. Copy the complete prompt below into a fresh ChatGPT Desktop conversation.

```
Connect to the Presentation Studio MCP and work in the currently open project.

First, read the Presentation Studio design contract, check the app status, inventory the authorized project
Resources, and inspect the installed ORNL Template Pack. Confirm that you can read the source content - not merely
filenames or metadata. If anything required is inaccessible, tell me exactly what must be shared or attached. Do not
invent missing content.

For a source-only project, read every required compatible text Resource completely with get_resource_text, inspect
stable layout IDs with get_template_layout_catalog, then call create_studio_presentation once with the complete
source-grounded deck plan. Create it without a starter PowerPoint; do not invent one.

Create a polished, editable, 16:9 ORNL presentation from the supplied source materials.

Content direction:
- Organize the material into a clear narrative using assertion-evidence slides.
- Give each slide one primary takeaway and supporting evidence.
- You may condense source prose, but preserve technical meaning, names, numbers, units, qualifications, and
attribution.
- Preserve approved or locked copy exactly. Do not introduce unsupported claims, data, diagrams, or conclusions.
- Infer routine structure and design choices. Ask only about genuine audience, technical, content-authority, or
approval ambiguities.

Design direction:
- Use Aptos and the current approved ORNL Template Pack.
- For a new title slide, use an approved ORNL title layout and edit only intended placeholders. Never alter its
artwork, marks, master, or layout.
- Make substantive whole-slide composition decisions using shared Studio recipes and compatible ORNL layouts. Do not
merely keep the source arrangement or make text smaller.
- Establish one deck-wide system for titles, spacing, alignment, figures, captions, tables, colors, and repeated
components.
- Use authorized Resource images only when they support the message. Preserve technical figures as
relationship-aware groups unless a verified editable reconstruction is clearer.
- Keep tables editable and readable; preserve meaning-bearing colors.

Workflow:
1. Develop the narrative and slide plan.
2. Create the presentation in the single central Studio HTML/CSS scene.
3. Build the complete editable PowerPoint candidate.
4. Inspect the PowerPoint-native contact sheet and every full-size candidate slide.
5. Run Found issues -> Fixing -> Rechecking original intent. Correct overflow, alignment, hierarchy, spacing,
tables, missing imagery, and message drift.
6. Do not call the presentation ready while any blocker or major visual issue remains.

Use your best design judgment and minimize routine questions. Leave the completed central design visible for my
review. Do not save or export the final PowerPoint until I explicitly request it.
```

Resource access

Turn on the single AI access switch. Presentation Studio automatically shares bounded extracted text for compatible documents/data, bounded previews for images, and metadata for every other embedded Resource. Presentation Studio 0.3.1 can create a new native Studio JSON deck from those sources and the ORNL Template Pack without a starter PowerPoint. PDF, legacy DOC/XLS, and raw original-file retrieval remain unavailable; convert them to a supported format or provide another approved source rather than inventing content.

Projects, encryption, and verification

Project formats

.pstudio	.pstudio-secure
ZIP-based self-contained package Canonical project JSON Immutable-by-hash Resources Portable without original file paths	Encrypts the complete .pstudio payload AES-256-GCM PBKDF2-SHA-256 with 250,000 iterations Password cannot be recovered

Encryption boundary Encrypted project files protect packaged JSON and Resources. They do not encrypt the external originals or separately exported PowerPoint, PDF, SVG, or PNG files.

Optional developer verification

```
npm run quality
npm run desktop:smoke
```

Expected result

Tests should pass, the repository scan should report no tracked client artifacts, Vite should produce a production bundle, and the Electron smoke check should save a valid renderer capture before exiting.

Troubleshooting

App reports that setup is missing

Run the one-line installer again. In a developer checkout, rerun the platform setup script from the repository root.

MCP says the app is unavailable

Open Presentation Studio first. Restart the desktop app if the local runtime descriptor is stale, then retry the MCP tool.

A deck needs manual review

Macros, embedded OLE objects, external relationships, uncertain templates, and ambiguous formatting intentionally stop automated cleanup. Preserve the source and resolve the finding in the app.